

Real-Time Dashboards & Alerting

React · FastAPI · PostgreSQL

Role	Full Stack Developer
Duration	1 day (take-home)
Stack	React · FastAPI · PostgreSQL
Format	GitHub repository + README

Overview

This assignment evaluates your ability to design a full-stack application with a real data model, a dashboard UI, and a separate alerting/analytics layer on top of standard CRUD. Build something that goes beyond a basic CRUD app — the emphasis is on architecture: how you separate the “detect something unusual” logic from core data operations, and how you reason about real-time updates.

Project A — Live Expense Tracker with Anomaly Alerts

Stack: React dashboard + FastAPI + basic rule/ML-based anomaly detection

- **What to build:** Log expenses, visualize spending, and flag unusual spikes via a Python service.
- **Architecture focus:** Event-driven thinking, separation of the analytics service from core CRUD.
- **Ideation angle (address in README):** How would you make this real-time? WebSockets vs. polling trade-offs?

Constraints & Ground Rules

Persistent storage	Use PostgreSQL (or another real relational database) — no in-memory-only data.
Separation of concerns	The anomaly/alerting logic must live in a distinct service/module from the CRUD API, not inline in the same route handlers.
Dashboard is real	The React dashboard must visualize actual data from the backend (charts/tables), not static mockups.
Concurrency-aware	Explain — and where feasible implement — how you avoid race conditions on concurrent writes to the same record.
Written trade-off analysis	Your README must directly answer the ideation angle questions for your chosen project.

Deliverables

1. Working application — React dashboard + FastAPI backend + PostgreSQL, runnable locally with clear setup/migration instructions.
2. Data model — A schema/ER diagram or equivalent description of your tables and relationships.

3. Alerting/analytics service — A clearly separated module or service implementing the anomaly detection or low-stock logic, with a description of the rule/model used.
4. Ideation answer — A written answer to the ideation angle (real-time strategy, WebSockets vs. polling).
5. Known limitations — A short, honest section on what's not handled (e.g. scale, edge cases).

Evaluation Criteria

Area	Weight	What We Look For
Data model & backend design	25%	Schema is sensible and normalized appropriately; API design is RESTful and consistent.
Separation of concerns	20%	Alerting/analytics logic is architecturally distinct from CRUD, not tangled into route handlers.
Frontend dashboard	20%	Dashboard accurately visualizes real data; state and loading/error handling are clean.
Real-time & concurrency reasoning	20%	Thoughtful, specific answer to the WebSockets vs. polling trade-off and (Project B) race-condition handling.
Code quality & docs	15%	Readable, organized code; README explains setup, schema, and design decisions.

Bonus (Optional)

- Implement real-time updates via WebSockets (not just describe the trade-off).
- Add a simple ML-based (rather than pure rule-based) anomaly/forecasting model.
- Add optimistic locking or a transaction-based approach to handle concurrent updates.
- Dockerize the full stack (API + DB) for one-command startup.

Submission Instructions

- Share a public or private GitHub repository link. If private, invite the interviewer's GitHub account (handle shared separately).
- Tag the final commit as v1.0.
- Include a short summary of what works and any known limitations in the README.
- Submit by replying to the assignment email with the repository link.